EXPERIMENT 09

DBSCAN Clustering – Python Code (Cell by Cell)

Aim: To perform density-based clustering on the Wine dataset using the DBSCAN algorithm, identify clusters and noise points, evaluate clustering performance, and visualize the results using PCA.

Dataset: Built-in Wine dataset from scikit-learn.

Cell 1: Fix warnings

```
import os
os.environ["OMP_NUM_THREADS"] = "1"

import warnings
warnings.filterwarnings("ignore")
```

Cell 2: Import libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
```

Cell 3: Load Wine dataset

```
data = load_wine()

X = data.data
y = data.target
```

Cell 4: Convert dataset into DataFrame

```
df = pd.DataFrame(X, columns=data.feature_names)

print("First Five Records:")
print(df.head())
```

Cell 5: Dataset Information

```
print("\nDataset Information:")
df.info()
```

Cell 6: Check missing values

```
print("\nMissing Values:")
print(df.isnull().sum())

print("\nDataset Shape:", df.shape)
```

Cell 7: Feature Scaling

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Cell 8: Apply DBSCAN algorithm

```
dbscan = DBSCAN(
    eps=2.2,
    min_samples=5
)

labels = dbscan.fit_predict(X_scaled)
```

Cell 9: Add cluster labels to DataFrame

```
df['Cluster'] = labels

print("\nDBSCAN Cluster Results:")
```

```
print(df.head(10))
```

Cell 10: Count number of clusters

```
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)

print("\nNumber of Clusters:", n_clusters)
```

Cell 11: Count noise points

```
n_noise = list(labels).count(-1)

print("Number of Noise Points:", n_noise)
```

Cell 12: Display cluster distribution

```
print("\nCluster Distribution:")
print(df['Cluster'].value_counts().sort_index())
```

Cell 13: Calculate Silhouette Score

```
mask = labels != -1

if len(set(labels[mask])) > 1:
    score = silhouette_score(
        X_scaled[mask],
        labels[mask]
    )
    print("\nSilhouette Score:", round(score, 4))
else:
    print("\nSilhouette Score cannot be calculated.")
```

Cell 14: Reduce dimensions using PCA

```
pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)
```

Cell 15: Visualize DBSCAN clusters

```
plt.figure(figsize=(8, 5))

for cluster in sorted(set(labels)):
    if cluster == -1:
        color = 'gray'
        name = 'Noise'
    else:
        color = None
        name = 'Cluster ' + str(cluster)

    points = labels == cluster

    plt.scatter(
        X_pca[points, 0],
        X_pca[points, 1],
        label=name,
        s=50,
        alpha=0.7,
        c=color
    )

plt.title("DBSCAN Clustering on Wine Dataset")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.grid()
plt.show()
```

Cell 16: Final Conclusion

```
print("\nConclusion:")
print("DBSCAN Clustering completed successfully.")
print("Number of Clusters:", n_clusters)
print("Number of Noise Points:", n_noise)

if len(set(labels[mask])) > 1:
    print("Silhouette Score:", round(score, 4))
```

Note: Run the cells in order, from Cell 1 to Cell 16. The exact number of clusters, noise points, and Silhouette Score are generated when the code is executed in your Python environment.